

RRSI: Regularized Recursive Self-Improvement of Agent Harnesses

Peng Xia, Rujun Han, Zifeng Wang, Yanfei Chen, Yufan Zhuang, Yoonho Lee, Chengsong Huang, Han Yu, Zhongying CuiZhu, Yifei Ming, Huaxiu Yao, Burak Gokturk, Tomas Pfister, Chen-Yu Lee

<https://arxiv.org/abs/2609.24972>

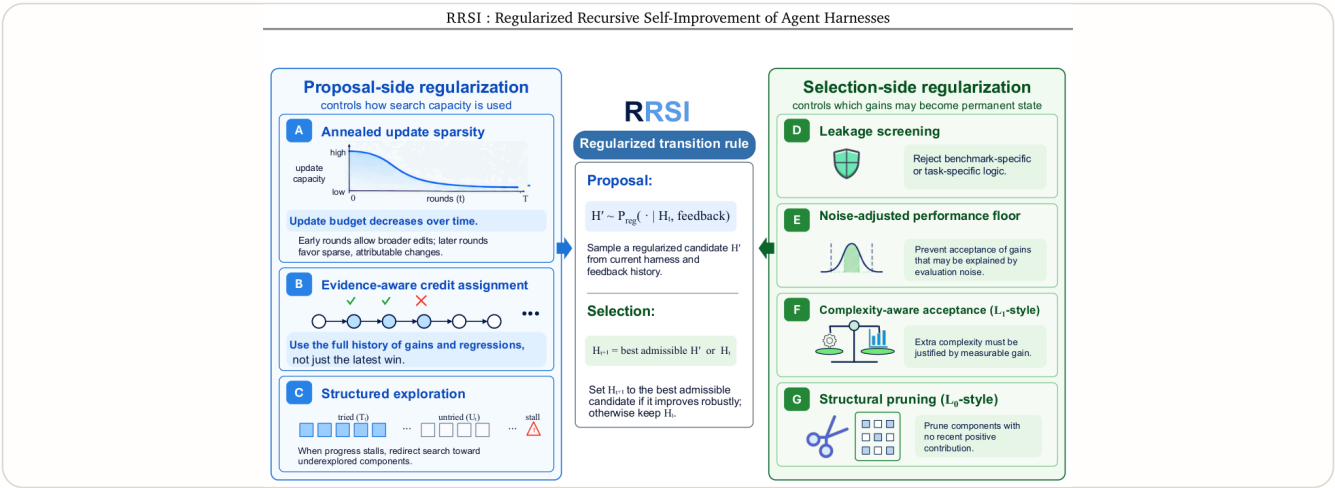


Figure 2 | Overview of RRSI. RRSI regularizes the search trajectory, not restricting the potential harness edit space: proposal-side constraints control how search capacity is used, while selection-side constraints control which measured improvements are allowed to become a permanent state.

Abstract

An LLM agent's capability is largely magnified by its harness, namely the prompts, control flow, tooling, memory, and context management surrounding the frozen backbone model. Recent methods increasingly automate this process by iteratively proposing and selecting component-wise edits of an agent harness, practically establishing a form of recursive self-improvement (RSI) at the agent-system level. However, such recursive evolution may overfit by memorizing the training tasks, showing large in-distribution gains that shrink or even vanish on out-of-distribution benchmarks. We introduce Regularized Recursive Self-Improvement of Agent Harnesses (RRSI), which incorporates the principles of regularizations into harness self-improvement by constraining the evolution candidate proposal and selection. The proposer operates with a temporally annealed budget, limiting how many edits a candidate can bundle, and it encourages unexplored trajectories based on evolution history. The selector is equipped with a critic and a pruner: the critic screens benchmark-specific proposals, while the pruner, removes changes that are too small, too expensive, or no longer useful. Together these constraints favor reusable agent mechanisms over benchmark-specific ones or even noises. Across eight benchmarks spanning coding, agentic workspace and engineering design tasks, RRSI gains up to 14.1 points on the split it evolves against and up to 4.7 points on the five out-of-distribution benchmarks, while producing a harness that runs on 30% fewer policy tokens than the unregularized evolution. Code is available at <https://github.com/google-research/rrsi> and project page is <https://regularized-rsi.com/>.

Task

Agent Harness Evolution Problem

- The paper defines an agent as a combination of a backbone policy and a harness managing prompts, control flow, tools, and memory.
- Task performance is the expected score on a task set, while policy-token cost is the expected number of tokens consumed.
- Harness evolution treats the harness as the optimization variable while keeping the backbone policy fixed.

Challenges

Overfitting Risks in Iterative Evolution

- Iterative harness evolution uses finite feedback, creating adaptive overfitting where evolve-set performance fails to generalize to unseen tasks.
- Search encodes benchmark-specific patterns and promotes candidates favored by evaluation noise, accumulating complexity without improving the agent mechanism.
- Existing approaches rely on benchmark scores without generalization terms, so reported gains often do not survive changes in the evaluation suite.

Proposed method

RRSI Regularization Framework Details

- RRSI anneals update capacity per round, uses evidence-aware credit assignment, and structures exploration.
- Leakage screening rejects candidates encoding task names or specific answers before full evaluation.
- Stability-aware acceptance uses a noise-adjusted floor to prevent selecting candidates due to stochastic variation.

Main results

Results, Gains, and Limitations

- RRSI improves six held-out splits by up to 4.7 points out of distribution using fewer policy tokens than unregularized evolution.
- The method gains 2.3 points on Harvey LAB and 1.8 points on SWE-bench Verified for in-distribution held-out tasks.
- Removing acceptance constraints raises the evolve-set score to 91.5 but drops out-of-distribution transfer from 43.6 to 41.0 and increases token cost.